

DATA 266 - Homework 4 Findings

Building a Mini GPT from Scratch (Multi-Head, Character-Level)

Personal Parameters

Parameter	Value
SID4	9486
SEED	9486
SLICE	486
HP_ID	0
CLS_A	6
CLS_B	0

SEED = 9486 was used for Python, NumPy, and PyTorch reproducibility throughout the notebook (weight initialization, DataLoader shuffling, and every sampling call). The remaining personal parameters are reported for consistency with the standing Assignment 1 instructions; this homework does not use SLICE, HP_ID, CLS_A, or CLS_B to select a dataset or hyperparameter arm.

Introduction

This homework builds a decoder-only, GPT-style Transformer entirely from PyTorch primitives (**nn.Linear**, **nn.LayerNorm**, **nn.GELU**, **nn.Embedding**) with no **nn.Transformer**, **nn.MultiheadAttention**, or HuggingFace transformer classes anywhere in the code. It extends the single-head demo model (**Demo_4_Mini_GPT.ipynb**) to 4-head masked self-attention, a sequence length of 128, and 4 decoder blocks, trains it on the given **shakespeare.txt** excerpt, and compares greedy, temperature, and top-k decoding. Every result in this report comes from the executed **HW4_Mini_GPT.ipynb** notebook.

Submission Contents

Part	What was completed
Part 1	Character-level tokenization of shakespeare.txt, a 128-length sliding-window dataset, and two decoded input/target ex
Part 2	Multi-head (4-head) masked self-attention built manually, a Transformer decoder block (attention + residual + LayerNo
Part 3	Adam optimizer, 6 training epochs, and a training-loss-vs-epoch plot.
Part 4	Greedy decoding, temperature sampling at T=0.5/1.0/1.5, and top-k sampling at k=5/20, all from the prompt "ROMEO:"
Part 5	A comparison of the three decoding methods grounded in the actual generated text.
Verification	Causal-mask row sums and zero future-attention weight, model output shape, divisibility of the hidden dimension by th

Part 1: Data Preparation

Dataset and Tokenization

shakespeare.txt (the Week 4 dataset, the opening of the balcony scene from *Romeo and Juliet*) was loaded directly from disk with no modification and no substitute dataset. Character-level tokenization produced a vocabulary of 49 distinct characters from 1,738 total characters.

Setting	Value
Total characters	1,738
Vocabulary size	49
Sequence length (seq_len)	128
Sliding-window training examples	1,609 (len(text) - seq_len - 1)
Input / target relationship	$x = \text{tokens}[t : t+128]$, $y = \text{tokens}[t+1 : t+129]$ (shifted by exactly 1 character)

Example Input/Target Pair (decoded)

Input x: "ROME0:\nBut, soft! what light through yonder window breaks?\nIt is the east, and Juliet is the su
Target y: "OME0:\nBut, soft! what light through yonder window breaks?\nIt is the east, and Juliet is the sur

Target y is exactly x shifted forward by one character, confirmed directly in the notebook with `torch.equal(x[1:], y[:-1])`.

Model Configuration

Setting	Value
Architecture	MiniGPT: token + positional nn.Embedding, N x DecoderBlock, final nn.LayerNorm, nn.Linear
Attention	Manual multi-head masked self-attention (no nn.MultiheadAttention); separate Q/K/V nn.Linear
Hidden dimension (d_model)	128
Number of heads (n_heads)	4 (head_dim = 128 / 4 = 32)
Number of decoder blocks (n_layers)	4
Feed-forward network	Linear(128 -> 512) -> GELU -> Linear(512 -> 128)
Total parameter count	822,321
Optimizer	Adam, lr = 3e-4
Batch size	32
Epochs	6
Loss	Cross-entropy over next-character logits, [B, T, vocab_size] vs. shifted [B, T] targets
Device	CPU in this run; CUDA selected automatically when available
Reproducibility	SEED = 9486 applied to random, NumPy, and torch (incl. CUDA generator)

Part 2: Multi-Head Masked Self-Attention

For an input of shape $[B, T, d_{\text{model}}]$, separate nn.Linear layers compute Q , K , and V , each reshaped to $[B, n_{\text{heads}}, T, \text{head_dim}]$ to split the hidden dimension into 4 independent attention subspaces. Attention scores ($Q @ K^T / \sqrt{\text{head_dim}}$, shape $[B, n_{\text{heads}}, T, T]$) are masked with a lower-triangular boolean mask before softmax, so position i 's score against every future position $j > i$ becomes $-\text{inf}$ and receives zero probability. The 4 heads' outputs are concatenated back to $[B, T, d_{\text{model}}]$ and passed through a final output projection.

Causal Mask Verification

Check	Result
Softmax row sums (per attention row)	min = 0.9999998808, max = 1.0000001192
Total attention weight on future positions ($j > i$)	0.0
Model output shape on a real training batch	$(32, 128, 49) = [\text{batch_size}, \text{seq_len}, \text{vocab_size}]$
Hidden dimension divisible by head count	$128 / 4 = 32$ (exact)

These checks were run directly on tensors in the notebook (not just visually inspected): every attention row sums to 1 as a valid softmax distribution, and the total attention mass placed on future tokens is exactly zero, confirming the causal mask blocks all future-position attention.

Part 3: Training

The model was trained for 6 epochs with the Adam optimizer ($\text{lr} = 3\text{e-}4$) and a batch size of 32, using cross-entropy loss between the model's $[B, T, \text{vocab_size}]$ logits and the one-character-shifted $[B, T]$ targets produced by the sliding-window dataset. Training loss decreased every epoch:

Epoch	Avg. Loss
1	2.9655
2	2.3758
3	2.1977
4	2.0952
5	1.9809
6	1.8303

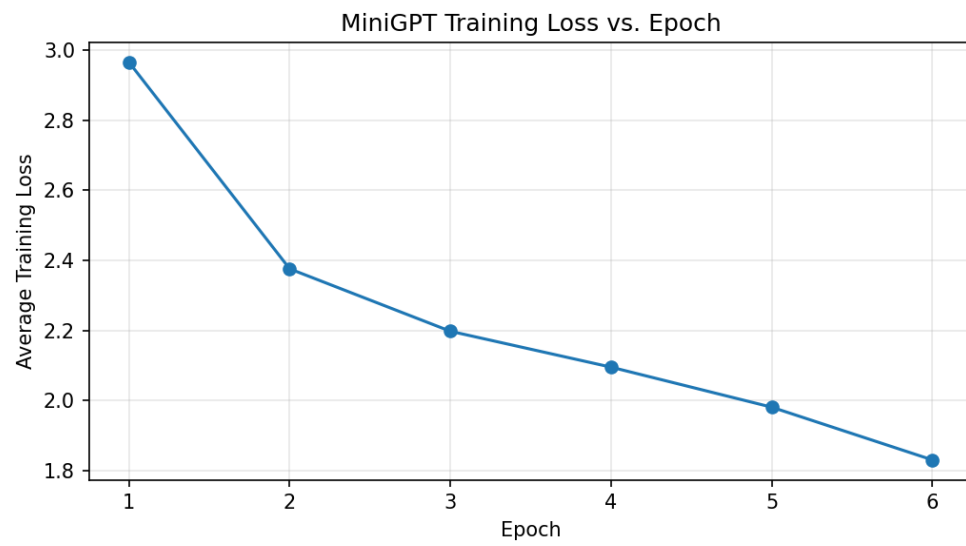


Figure 1. Training loss vs. epoch (MiniGPT, 6 epochs, batch size 32).

Part 4: Sampling (prompt = "ROMEO:")

All samples below generate 300 new characters from the same trained model and the same prompt, re-seeding with SEED = 9486 before each call so results are reproducible.

Greedy Decoding

```
ROMEO:
Romeomeeo, theou nat thy nathamer that namer nd nar na nar whe
Or nor nor nathe
O nor nathame
Whathath tha nath name
Whathame n nor name
O nor na nor nor na na nor a name
Wha na a na na na a a name
OMor nor nor ame
OMor name
OMor name
OMor ame
OMEOMEO:
O:
O:
O:
O:
O ame, nor nor name, name, nor nor
```

Temperature Sampling

T = 0.5:

```
ROMEO:
Roweat the thy theerdirt nof, ndous
Shout far not naind the fat ner the she nait n,
Ar ther ithat ind nat inin is ir ise isis s s?
Wher n, s sher ind sher isher ansur she is;
An.
I mour anor e s nout ar mall ano an.
By amy ne nonksther amer ame athe, be anguswishe ar at ictat siny t ind ino inot;
S
```

T = 1.0:

```
ROMEO:
Rowear the reou meomen therf, owha!
Sh Rou bem, ornt'aicNo? fat nere tilllllll
ROM;
BAng e oreoRowh a!
MESome RO, k treome, I t;;
Car thou be, s:
RO:
O:
I stho, nakeso.Dear to! bere ereaks there an bt t wot sathat t.
Benad mer nonksthese;
By bulFULI t'leWhern w s
O f, asad tat'sharern ves niWind yie
```

T = 1.5:

```
ROMEO:
now ais hharehin.
Ek n t nof, r whone.
Rome, m,?
DnI'
Ca houetomeewe theaDCScAmy ;
BARffo, ndow cl !
```

MySare inolllll whe ber nd a?

?EBgur m, s:

RO

Har Iustis, nMu

JU.Dofr cazever! ereaks Rowol and?t ULito sathar berernod aned.

Okst Ron'n beecerULIzt'beW; IEOW s

Ccaleas, htailsilyet; ves n WUETJyie

Top-k Sampling

k = 5:

ROMEO:
Rowear the allllllean, t be meow t Reomeove,

ROMEOM'dow beofor bear bulllllll'de I n beald Rore be neort Re R0,

Roreomeareoreor beart bureeort ticr wineo.
Ro, n t ther t;
Someoreo t tichanareor thout t beou t n t t t onat or t fard orenathy ame,
Tanthe;
Ber me, me ar at, me;
By ar tor thy, foulonor

k = 20:

ROMEO:
Rowear the reou meomen therf, owhas
She ou ben, ornt'ad thouet inere tilllll be nd,
Angulo Roth t aieomatom inow the, ce mer n mar at ber bar Ror I'd, I sheo,
Anansmer fr id r ce!
Tather there llur t theoner thar t.
Benad me spe!
By Ronanor bulfainy mbe angumw amacaleast htat silre,
He!
By ffove;
S

Decoding Comparison

With only 6 epochs over a ~1,700-character excerpt, none of the six samples read as fluent Shakespeare, but the differences between decoding methods are still clearly visible. **Greedy decoding** fell into a tight repetition loop almost immediately ("nor nor nathe", "O nor nathame", "nor name" repeated many times) - the classic failure mode of always taking argmax on an undertrained model. **Temperature sampling** showed a clear, increasing progression from T=0.5 (still some repeated letters, closest to English-like fragments) to T=1.0 (more erratic capitalization and punctuation) to T=1.5 (the most chaotic sample of the six, with rare characters appearing mid-word). **Top-k sampling** at k=5 still showed visible looping ("bulllllll'de", "bureeort"), staying close to greedy's repetition problem, while k=20 was noticeably more varied, closely resembling the T=1.0 sample since both draw from a wide character pool.

Most coherent output: T=0.5 and k=5 stayed closest to recognizable English letter patterns and word starts, since restricting the distribution keeps the model near its most confident (and most training-text-like) predictions. **Most diverse output:** T=1.5 produced the widest range of characters, capitalization, and punctuation of all six samples, with k=20 a clear second - consistent with what each method does to the sampling distribution rather than an independent claim beyond these printed samples.

Final Findings

The from-scratch multi-head masked self-attention, decoder block, and full MiniGPT model satisfy every architectural constraint of the assignment: 4 heads with $128/4=32$ -dimensional heads, 4 decoder blocks, learned token and positional embeddings, and a causal mask verified directly against tensors. Training loss decreased every epoch (2.97 to 1.83), but 1,609 overlapping 128-character windows from a single short excerpt, trained for only 6 epochs, was not enough to produce fluent generated text - every Part 4 sample is visibly undertrained, which this report states directly rather than overstating the model's quality. The clearest,

most reproducible pattern across decoding methods is that greedy and low-temperature/low-k sampling repeat themselves, while higher temperature and larger k trade that repetition for more diverse but less structured output - exactly the behavior each method is designed to produce. No `nn.Transformer`, `nn.MultiheadAttention`, or HuggingFace transformer classes were used anywhere in the implementation.